Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych
Inżynieria Oprogramowania

RoboFleet

Checkpoint 2: stany i aktywności systemu

Diagramy stanów kluczowych obiektów biznesowych i diagramy aktywności kluczowych funkcjonalności

Anna Granos
Mikołaj Zawrotniak
Patrick Fongang
Bartek Dźwigała

Październik 2026

1 Wprowadzenie

Dokument zawiera diagramy stanów pięciu kluczowych obiektów biznesowych systemu RoboFleet oraz diagramy aktywności pięciu kluczowych, powtarzalnych funkcjonalności. Diagramy stanów obejmują przejścia standardowe, przejścia nietypowe i stany specjalne; diagramy aktywności obejmują ścieżkę główną i kluczowe ścieżki alternatywne. Nazwy klas, metod i wartości enumeracji są zgodne z diagramami klas z Checkpointu 1 (CD-1 do CD-3).

1.1 Obiekty biznesowe

Diagram	Obiekt (klasa)	Atrybut stanu	Dlaczego kluczowy
SM-1	Przebieg symulacji (`SimulationRun`)	`status: RunStatus`	Główna jednostka pracy klienta; od stanu zależy, które polecenia API są dozwolone.
SM-2	Zlecenie transportowe (`TransportOrder`)	`status: OrderStatus`	Podstawowa jednostka biznesowa; z jego stanów liczone są statystyki (czas realizacji, przepustowość).
SM-3	Robot w trakcie przebiegu (`RobotAgent`)	`state: RobotState`	Realizuje zlecenia; stany ruchu i oczekiwania decydują o braku kolizji i o wykrywaniu zastoju.
SM-4	Magazyn (`Warehouse`)	`status` (nowy, zamiast `archived`)	Konfiguracja, na której opiera się każdy przebieg; tylko poprawny magazyn może być użyty.
SM-5	Konto użytkownika (`User`)	`isActive`, `role`	Określa dostęp do wszystkich pozostałych funkcji.

1.2 Funkcjonalności

Diagram	Funkcjonalność	Przypadek użycia (CP1)	Częstość
AD-1	Krok symulacji (tick)	Wykonaj krok symulacji	co `tickIntervalMs` w każdym trwającym przebiegu
AD-2	Uruchomienie przebiegu	Uruchom symulację	przy każdym przebiegu
AD-3	Dodanie zlecenia transportowego	Dodaj zlecenie	wielokrotnie dla każdego przebiegu
AD-4	Projektowanie i zapis magazynu	Edytuj układ magazynu, Zapisz magazyn	przy każdej zmianie konfiguracji
AD-5	Podgląd przebiegu na żywo i ponowne połączenie	Obserwuj symulację, Połącz ponownie	przez cały czas trwania przebiegu

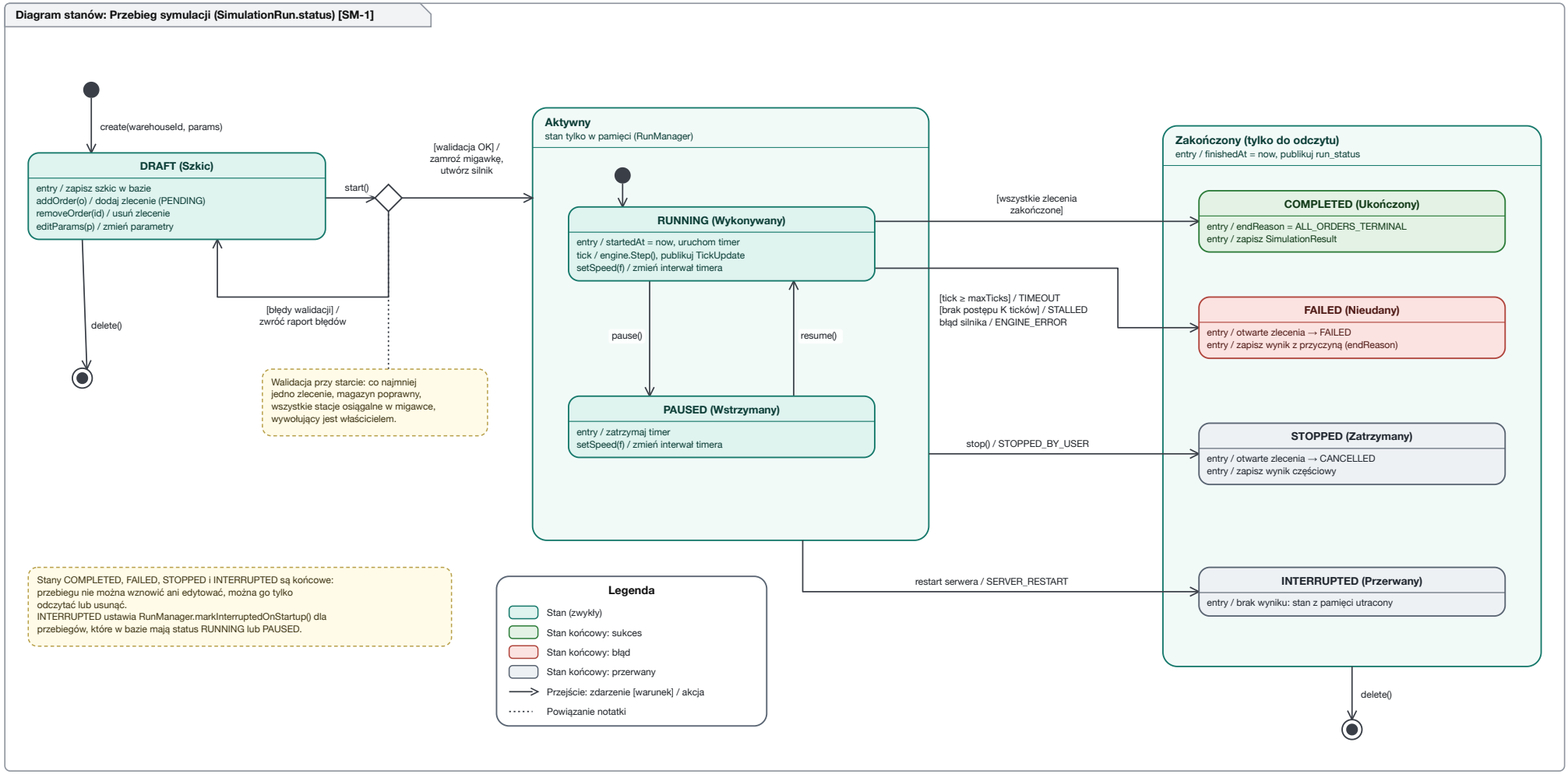
2 Diagramy stanów

2.1 Przebieg symulacji (SM-1)

Przebieg powstaje jako szkic, w którym klient dodaje zlecenia i ustawia parametry. Polecenie *start()* waliduje przebieg na zamrożonej migawce magazynu; w razie błędów przebieg pozostaje szkicem. Stan aktywny (RUNNING, PAUSED) istnieje wyłącznie w pamięci RunManager, a w bazie zapisywany jest tylko status. Przebieg kończy się jednym z czterech stanów końcowych, które są tylko do odczytu; każdy z nich zapisuje przyczynę zakończenia (`EndReason`).

Przejścia nietypowe

Przejście	Zdarzenie [warunek] / akcja	Uzasadnienie
DRAFT → DRAFT	start() [błędy walidacji] / zwróć raport	Brak zleceń, niepoprawny magazyn lub stacja nieosiągalna w migawce. Klient poprawia szkic i ponawia start.
RUNNING → FAILED	[tick ≥ maxTicks] / TIMEOUT	Ochrona przed przebiegiem, który nigdy się nie kończy.
RUNNING → FAILED	[brak postępu przez K ticków] / STALLED	Zakleszczenie robotów: żaden robot się nie porusza i żadne zlecenie nie zmienia stanu.
RUNNING → FAILED	błąd silnika / ENGINE_ERROR	SafetyMonitor wykrył konflikt lub wystąpił wyjątek; wynik byłby niewiarygodny, więc przebieg jest przerywany.
Aktywny → STOPPED	stop() / STOPPED_BY_USER	Dozwolone zarówno z RUNNING, jak i z PAUSED; otwarte zlecenia są anulowane.
Aktywny → INTERRUPTED	restart serwera / SERVER_RESTART	Wykrywane przy starcie aplikacji (`markInterruptedOnStartup()`) dla przebiegów zapisanych jako RUNNING lub PAUSED.



Rysunek SM-1. Diagram stanów przebiegu symulacji (SimulationRun.status)

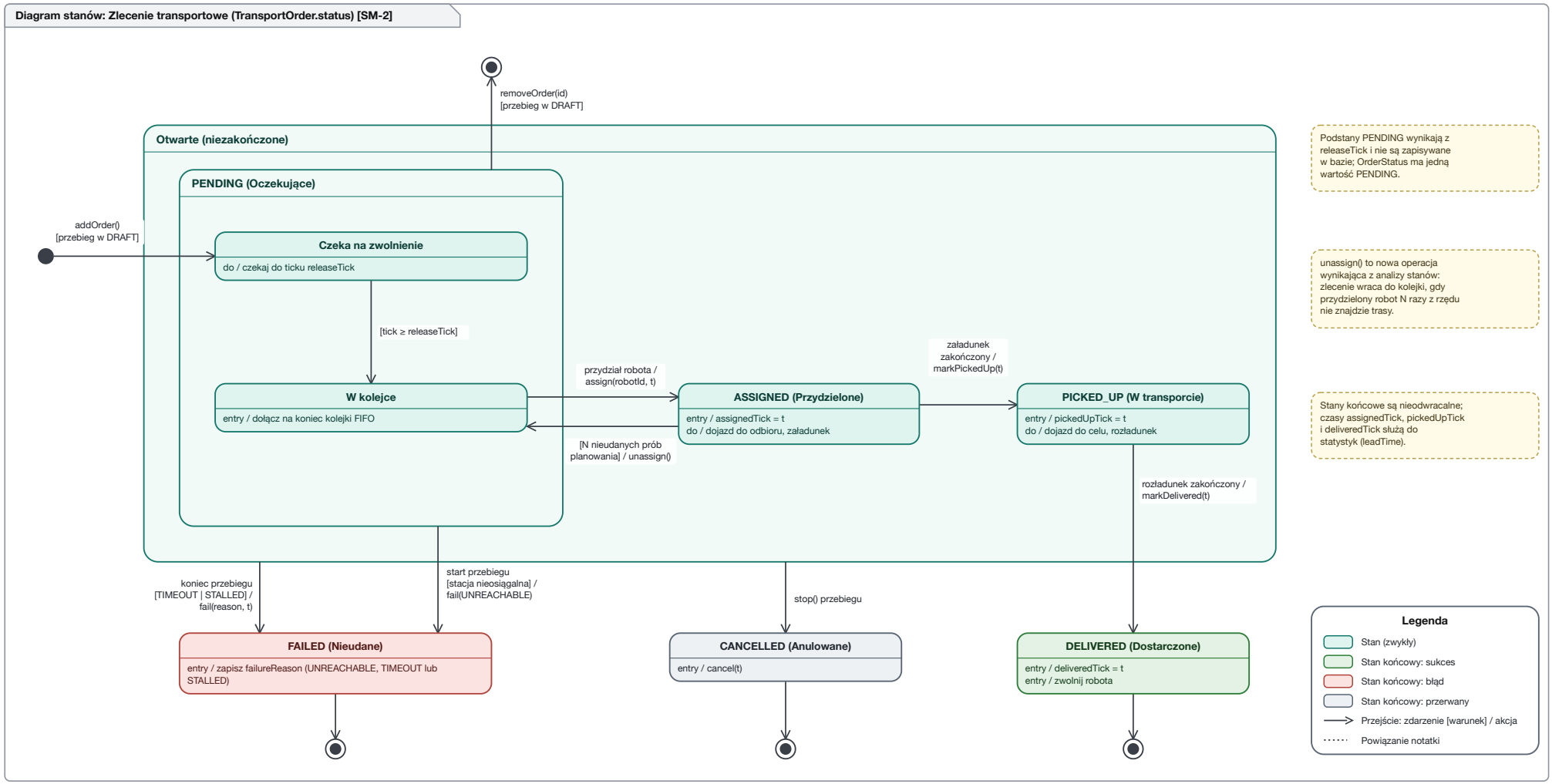
2.2 Zlecenie transportowe (SM-2)

Zlecenie jest dodawane do przebiegu w stanie DRAFT. Stan PENDING ma dwa podstany wyliczane z `releaseTick`: zlecenie czeka na swój tick zwolnienia, a następnie trafia do kolejki FIFO. Po przydziale robot dojeżdża do punktu odbioru i ładuje towar (ASSIGNED), przewozi go (PICKED_UP) i rozładowuje (DELIVERED). Stan złożony *Otwarte* pozwala opisać jednym przejściem skutki zakończenia przebiegu dla wszystkich niezakończonych zleceń.

Przejścia nietypowe

Przejście	**Zdarzenie [warunek] / akcja**	**Uzasadnienie**
ASSIGNED → PENDING	[N nieudanych prób planowania] / unassign()	Robot nie może dojechać do punktu odbioru (np. zablokowany przez inne roboty). Zlecenie wraca do kolejki i może trafić do innego robota.
PENDING → FAILED	start przebiegu [stacja nieosiągalna] / fail(UNREACHABLE)	Magazyn zmienił się po dodaniu zlecenia; osiągalność jest sprawdzana ponownie na migawce.
Otwarte → FAILED	koniec przebiegu [TIMEOUT | STALLED] / fail(reason, t)	Zlecenia niezrealizowane do końca nieudanego przebiegu.
Otwarte → CANCELLED	stop() przebiegu	Odróżnia rezygnację użytkownika od błędu systemu w statystykach.
PENDING → (usunięte)	removeOrder(id) [przebieg w DRAFT]	Zlecenie z szkicu jest usuwane fizycznie, bo nie brało udziału w symulacji.

Zlecenie w stanie PICKED_UP nie może wrócić do kolejki: robot z towarem czeka na trasę aż do skutku, a ewentualne zakleszczenie kończy przebieg statusem STALLED.



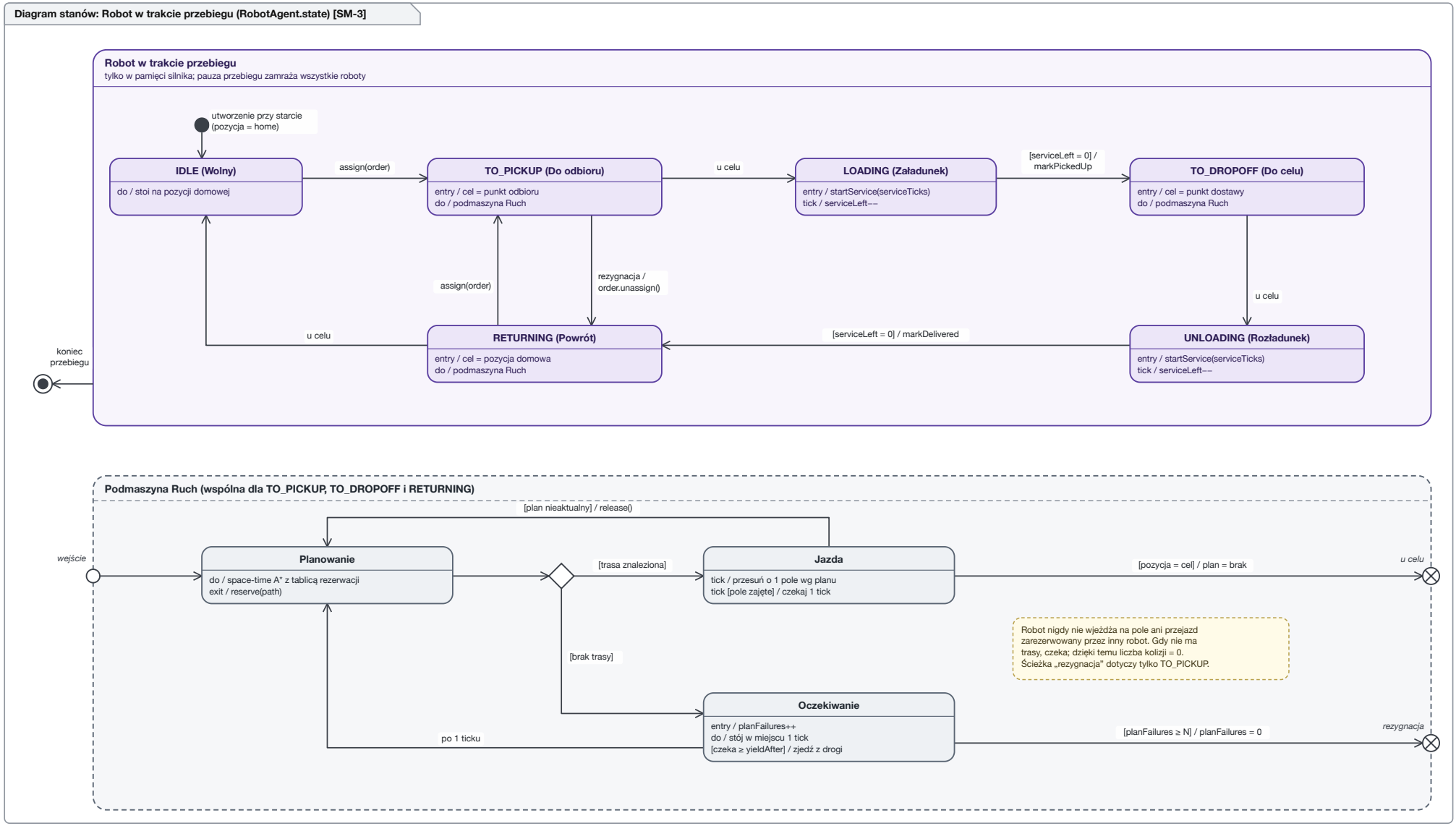
Rysunek SM-2. Diagram stanów zlecenia transportowego (TransportOrder.status)

2.3 Robot w trakcie przebiegu (SM-3)

Stan robota istnieje tylko w pamięci silnika symulacji i jest tworzony przy starcie przebiegu. Cykl pracy to: IDLE, dojazd do odbioru, załadunek, dojazd do celu, rozładunek i powrót na pozycję domową. Trzy stany jazdy (TO_PICKUP, TO_DROPOFF, RETURNING) korzystają ze wspólnej podmaszyny *Ruch*, która opisuje planowanie trasy, jazdę i oczekiwanie. Podmaszyna kończy się w jednym z dwóch punktów wyjścia: *u celu* albo *rezygnacja*.

Przejścia nietypowe

Przejście	**Zdarzenie [warunek] / akcja**	**Uzasadnienie**
RETURNING → TO_PICKUP	assign(order)	Robot wracający do bazy jest wolny, więc może od razu przyjąć zlecenie, bez dojazdu na pozycję domową.
TO_PICKUP → RETURNING	rezygnacja / order.unassign()	Robot N razy z rzędu nie znalazł trasy; zlecenie wraca do kolejki (SM-2).
Jazda → Planowanie	[plan nieaktualny] / release()	Rezerwacje innych robotów zmieniły się; robot zwalnia swoje rezerwacje i planuje od nowa.
Oczekiwanie (wewnętrzne)	[czeka ≥ yieldAfter] / zjedź z drogi	Robot blokujący przejazd ustępuje na sąsiednie wolne pole, co zapobiega zakleszczeniu w wąskich korytarzach.



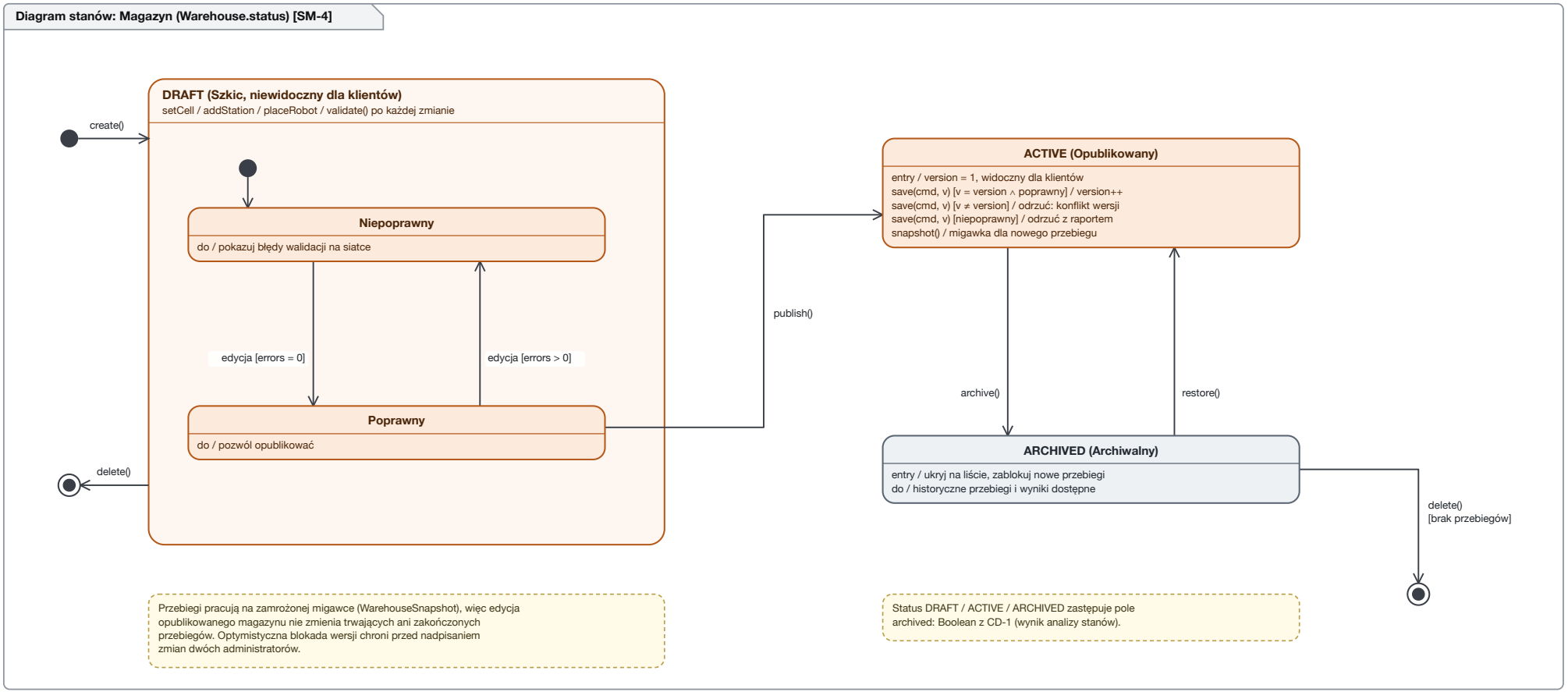
Rysunek SM-3. Diagram stanów robota w trakcie przebiegu (RobotAgent.state) z podmaszyną Ruch

2.4 Magazyn (SM-4)

Analiza stanów pokazała, że pojedyncze pole `archived: Boolean` z CD-1 nie wystarcza: klient nie powinien widzieć magazynu w trakcie projektowania, a przebieg można utworzyć tylko na poprawnym, opublikowanym magazynie. Wprowadzono status DRAFT / ACTIVE / ARCHIVED. Szkic ma podstany *Niepoprawny* i *Poprawny*, wyliczane przez `validate()` po każdej zmianie. Edycja opublikowanego magazynu jest bezpieczna, bo przebiegi korzystają z zamrożonej migawki; przed nadpisaniem cudzych zmian chroni numer wersji.

Przejścia nietypowe

Przejście	Zdarzenie [warunek] / akcja	Uzasadnienie
ACTIVE → ACTIVE	save(cmd, v) [v ≠ version] / odrzuć	Dwóch administratorów edytuje ten sam magazyn; zapis starszej wersji kończy się błędem 409 (AD-4).
ACTIVE → ACTIVE	save(cmd, v) [niepoprawny] / odrzuć z raportem	Opublikowany magazyn nigdy nie staje się niepoprawny.
ARCHIVED → (usunięty)	delete() [brak przebiegów]	Magazyn z historią przebiegów nie może zostać usunięty, bo wyniki odwołują się do niego.



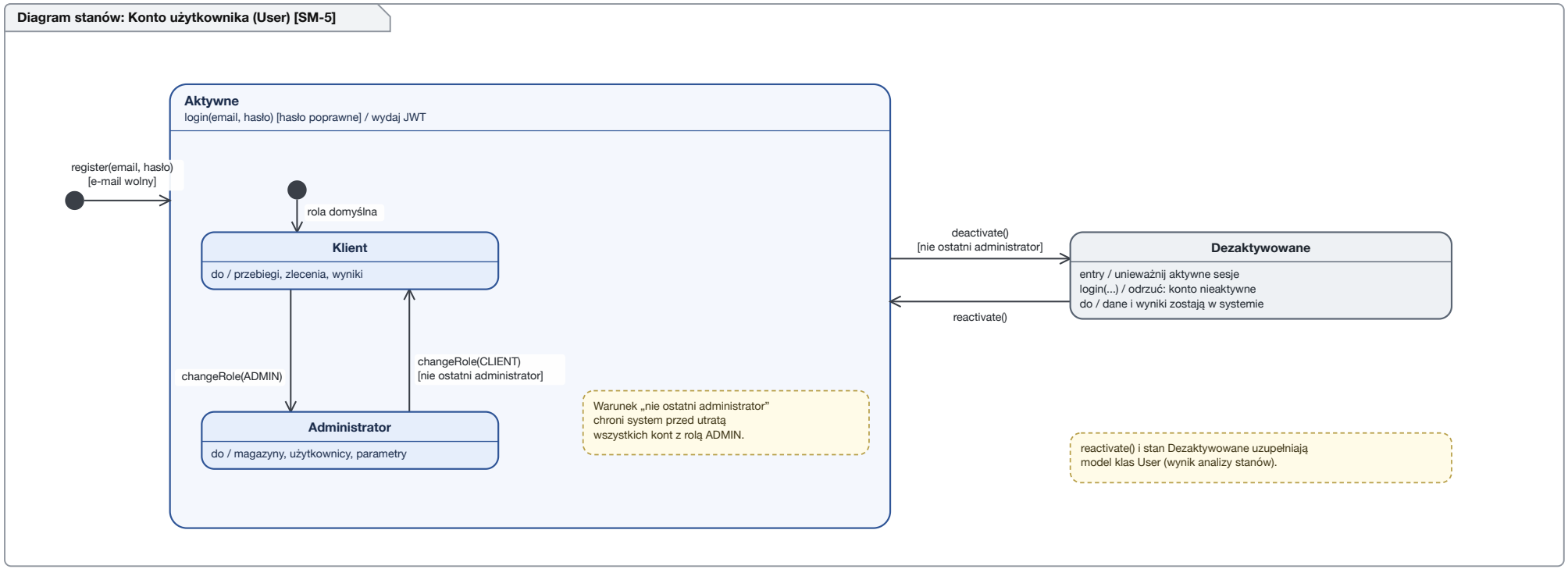
Rysunek SM-4. Diagram stanów magazynu (Warehouse.status)

2.5 Konto użytkownika (SM-5)

Konto po rejestracji jest aktywne z rolą Klient. Administrator może zmienić rolę lub dezaktywować konto. Konta nie są usuwane fizycznie, ponieważ są właścicielami przebiegów i wyników; dlatego diagram nie ma stanu końcowego.

Przejścia nietypowe

Przejście	Zdarzenie [warunek] / akcja	Uzasadnienie
Administrator → Klient	changeRole(CLIENT) [nie ostatni administrator]	System musi zawsze mieć co najmniej jedno aktywne konto administratora.
Aktywne → Dezaktywowane	deactivate() [nie ostatni administrator]	Jak wyżej; dodatkowo token JWT jest odrzucany przy kolejnym żądaniu, bo AccessPolicy sprawdza `isActive`.
Dezaktywowane (wewnętrzne)	login(...) / odrzuć	Poprawne hasło nie wystarcza; komunikat nie ujawnia, czy hasło było poprawne.



Rysunek SM-5. Diagram stanów konta użytkownika (User)

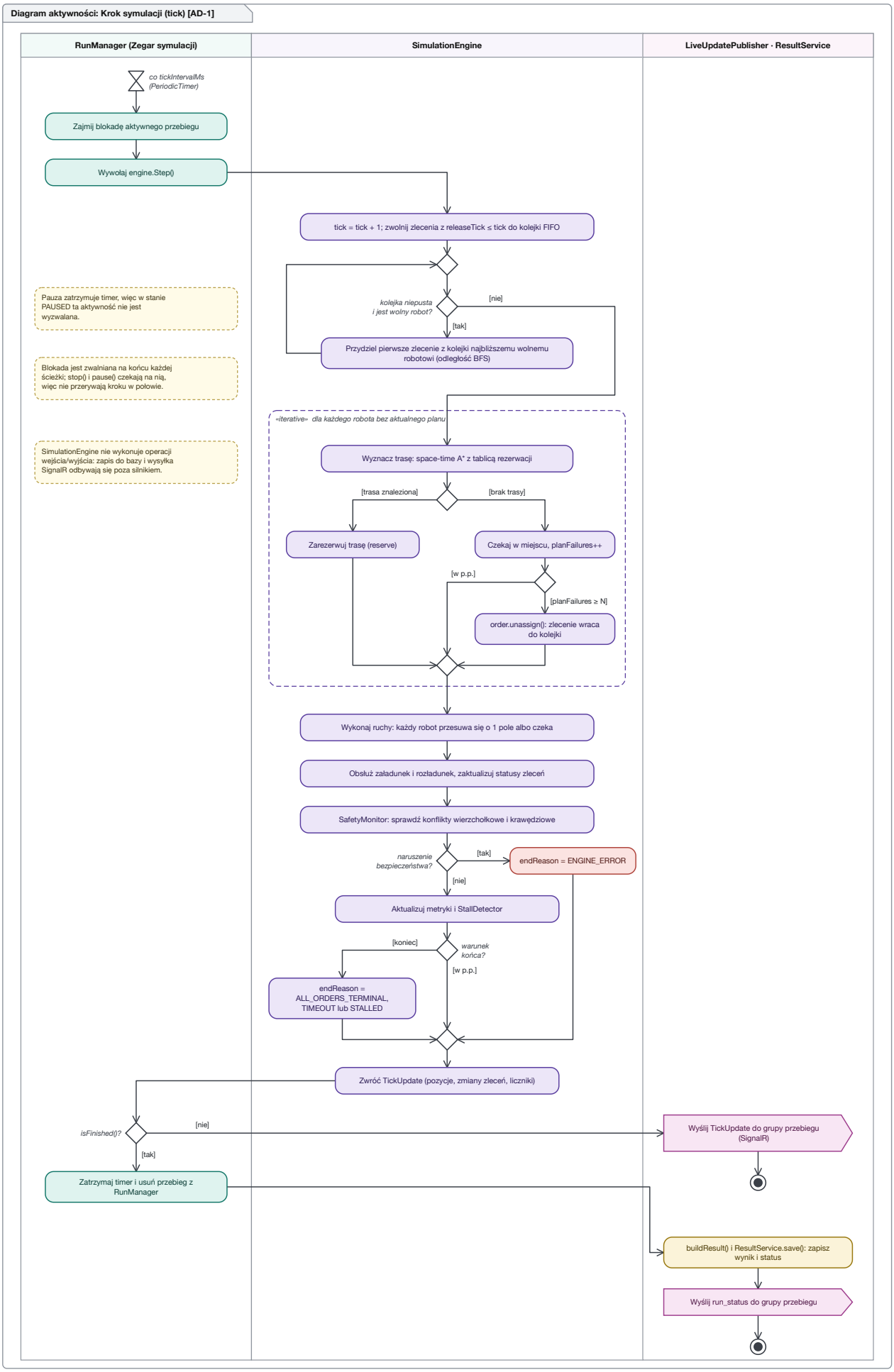
3 Diagramy aktywności

3.1 Krok symulacji (AD-1)

Najczęściej wykonywana funkcjonalność systemu. Zegar symulacji (PeriodicTimer w RunManager) wyzwala krok co `tickIntervalMs` dla każdego przebiegu w stanie RUNNING. RunManager zajmuje blokadę przebiegu i wywołuje `engine.Step()`. Silnik zwiększa licznik ticków, zwalnia zlecenia, przydziela je zachłannie (FIFO, najbliższy wolny robot według odległości BFS), planuje trasy dla robotów bez aktualnego planu, wykonuje ruchy i operacje załadunku, sprawdza bezpieczeństwo i warunki końca. Wynik kroku (`TickUpdate`) jest wysyłany do przeglądarek albo, jeśli przebieg się zakończył, RunManager zatrzymuje timer i zapisuje wynik. Roboty są obsługiwane w stałej kolejności (według identyfikatora), więc te same dane wejściowe zawsze dają ten sam wynik.

Ścieżki alternatywne

Warunek	Reakcja systemu
Brak trasy dla robota	Robot czeka w miejscu, `planFailures` rośnie; planowanie jest ponawiane w kolejnym ticku.
`planFailures ≥ N`	`order.unassign()`: zlecenie wraca do kolejki, robot przechodzi do RETURNING.
Naruszenie bezpieczeństwa	`endReason = ENGINE_ERROR`, przebieg kończy się statusem FAILED.
Warunek końca	ALL_ORDERS_TERMINAL (COMPLETED) albo TIMEOUT lub STALLED (FAILED); zapis wyniku i wysłanie `run_status`.



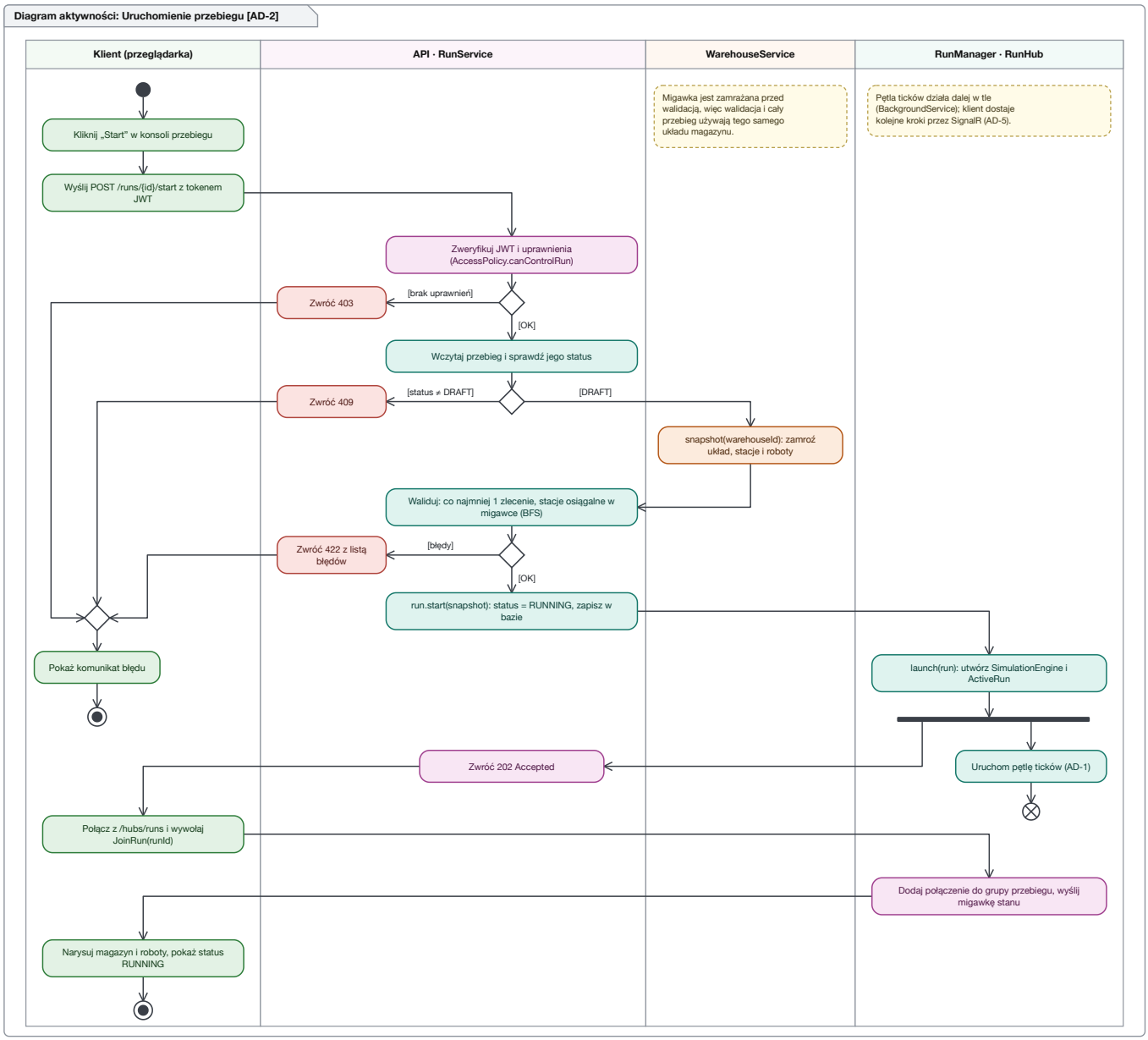
Rysunek AD-1. Diagram aktywności kroku symulacji

3.2 Uruchomienie przebiegu (AD-2)

Klient wysyła polecenie startu przez REST. API sprawdza uprawnienia i status przebiegu, WarehouseService zamraża migawkę magazynu, a RunService waliduje przebieg na tej migawce. Po udanej walidacji status zmienia się na RUNNING, a RunManager tworzy silnik. Rozwidlenie pokazuje dwa współbieżne przepływy: pętla ticków startuje w tle, a klient dostaje odpowiedź 202, łączy się z hubem SignalR i otrzymuje migawkę stanu do narysowania.

Ścieżki alternatywne

Warunek	Reakcja systemu
Brak uprawnień	403; klient pokazuje komunikat.
Status inny niż DRAFT	409 (np. podwójne kliknięcie „Start”); stan przebiegu bez zmian.
Błędy walidacji	422 z listą błędów (brak zleceń, stacja nieosiągalna, niepoprawny magazyn); przebieg pozostaje szkicem.



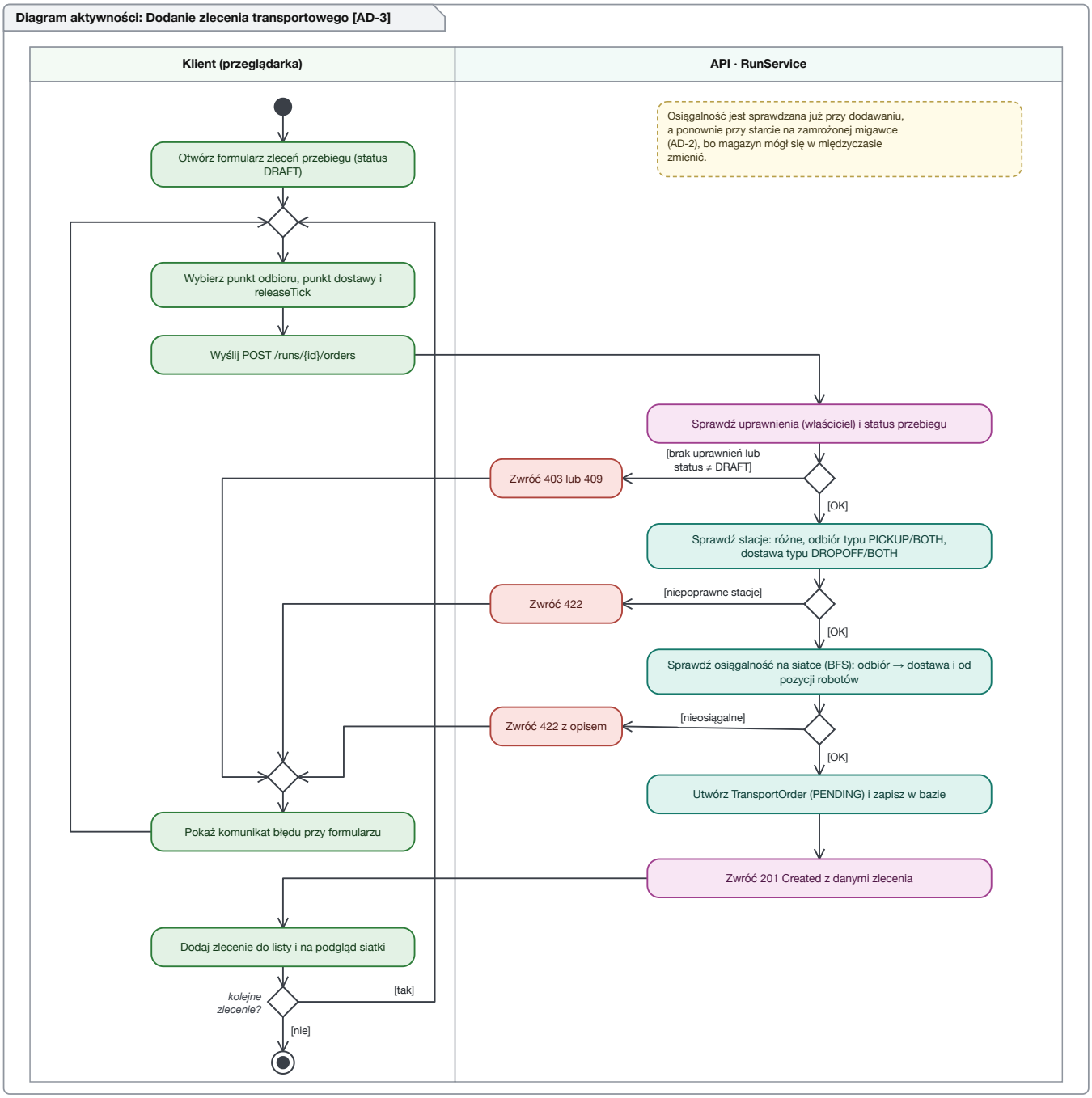
Rysunek AD-2. Diagram aktywności uruchomienia przebiegu

3.3 Dodanie zlecenia transportowego (AD-3)

Klient wybiera punkt odbioru, punkt dostawy i opcjonalny tick zwolnienia. Serwer sprawdza kolejno uprawnienia i status przebiegu, typy stacji oraz osiągalność na siatce (BFS). Poprawne zlecenie jest zapisywane w stanie PENDING. Pętla pozwala dodać kolejne zlecenia; każdy błąd wraca do formularza, a klient może poprawić dane i spróbować ponownie.

Ścieżki alternatywne

Warunek	Reakcja systemu
Brak uprawnień lub przebieg nie w DRAFT	403 lub 409.
Niepoprawne stacje	422: punkty identyczne albo stacja niewłaściwego typu (PICKUP, DROPOFF, BOTH).
Punkt nieosiągalny	422 z opisem, którego odcinka nie da się przejechać.



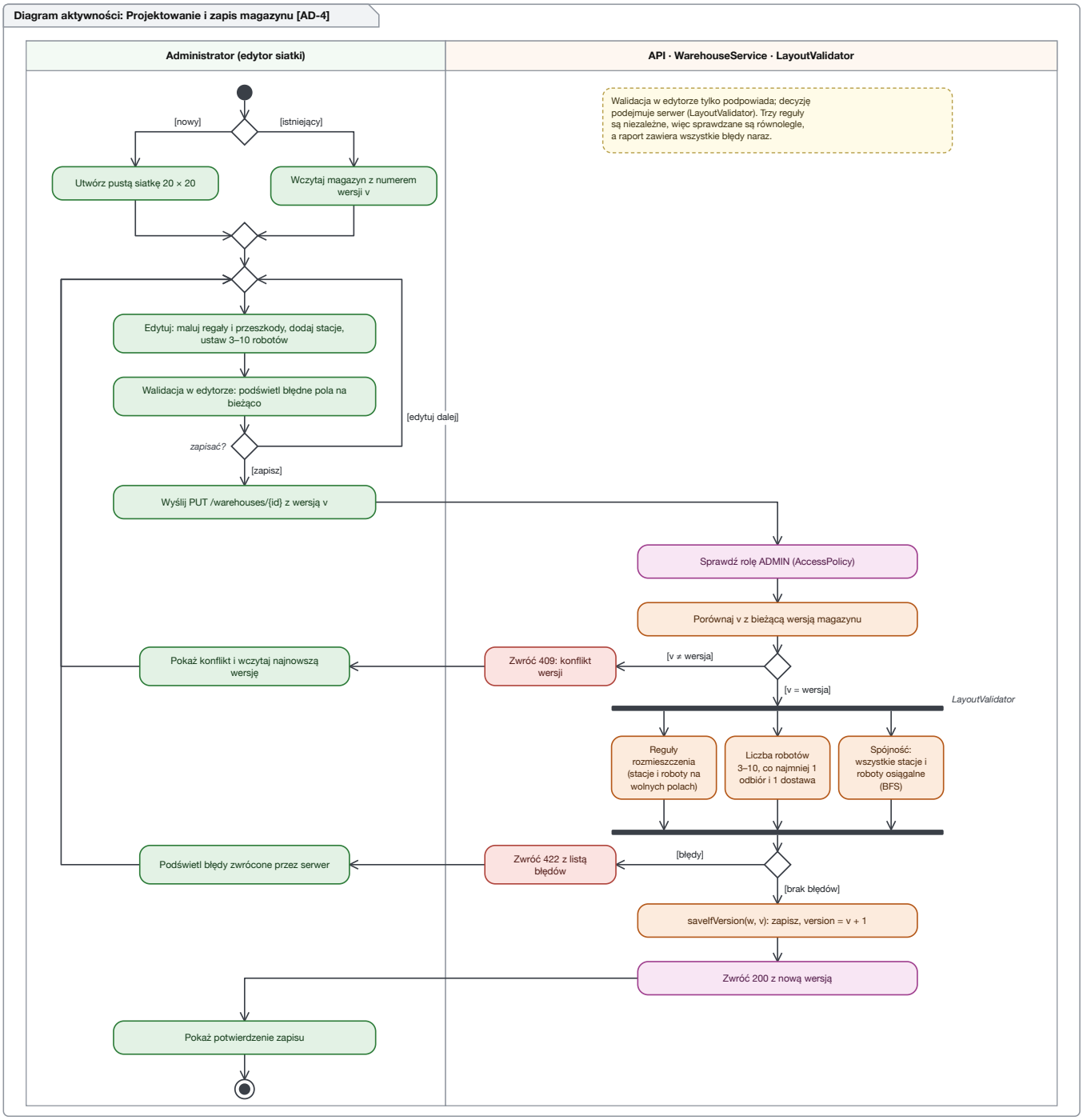
Rysunek AD-3. Diagram aktywności dodania zlecenia transportowego

3.4 Projektowanie i zapis magazynu (AD-4)

Administrator tworzy nową siatkę albo wczytuje istniejący magazyn wraz z numerem wersji. Edycja przebiega w pętli z walidacją w przeglądarce, która tylko podpowiada. Przy zapisie serwer sprawdza rolę i wersję, a następnie LayoutValidator sprawdza równolegle trzy niezależne grupy reguł (rozwidlenie i złączenie), dzięki czemu raport zawiera wszystkie błędy naraz. Zapis używa optymistycznej blokady `saveIfVersion`.

Ścieżki alternatywne

Warunek	Reakcja systemu
Wersja nieaktualna	409; edytor wczytuje najnowszą wersję i administrator wraca do edycji.
Błędy walidacji	422 z listą błędów; edytor podświetla błędne pola.
Administrator rezygnuje z zapisu	Pętla edycji trwa dalej; nic nie jest wysyłane do serwera.



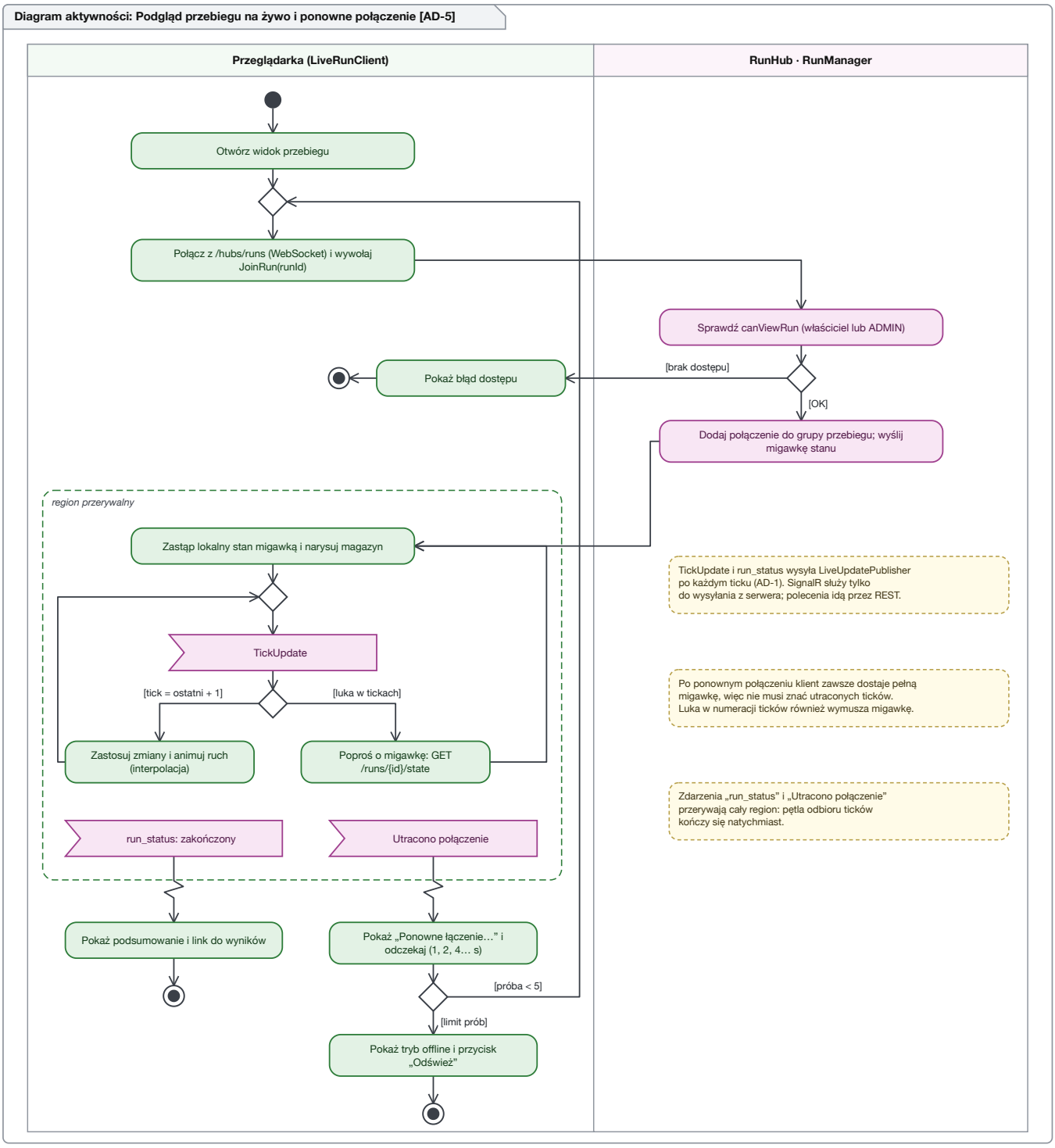
Rysunek AD-4. Diagram aktywności projektowania i zapisu magazynu

3.5 Podgląd przebiegu na żywo i ponowne połączenie (AD-5)

Przeglądarka łączy się z hubem SignalR i dołącza do grupy przebiegu. Po sprawdzeniu uprawnień dostaje migawkę stanu, a następnie w regionie przerywalnym odbiera kolejne `TickUpdate` i animuje ruch robotów. Region przerywają dwa zdarzenia: zakończenie przebiegu oraz utrata połączenia. Po utracie połączenia klient ponawia próbę z rosnącym odstępem i po udanym połączeniu zawsze dostaje pełną migawkę.

Ścieżki alternatywne

Warunek	Reakcja systemu
Brak dostępu do przebiegu	Połączenie odrzucone; klient pokazuje błąd dostępu.
Luka w numeracji ticków	Klient pobiera migawkę przez REST i kontynuuje od niej.
Zakończenie przebiegu	Podsumowanie i link do wyników.
Utrata połączenia	Ponowne łączenie co 1, 2, 4… s; po 5 nieudanych próbach tryb offline z przyciskiem „Odśwież”.



Rysunek AD-5. Diagram aktywności podglądu przebiegu na żywo